

Firecrawl Manual

A complete guide to installing, configuring, and using Firecrawl — the API for turning any website into clean, LLM-ready data.

Table of Contents

1. [Introduction](#)
2. [Getting an API Key](#)
3. [Installation](#)
4. [Authentication](#)
5. [Core Endpoints](#)
 - [5.1 Scrape](#)
 - [5.2 Crawl](#)
 - [5.3 Map](#)
 - [5.4 Search](#)

- [5.5 Extract](#)
 - [5.6 Interact / Agent](#)
 - 6. [Output Formats](#)
 - 7. [Webhooks & Async Delivery](#)
 - 8. [Credits, Pricing & Rate Limits](#)
 - 9. [MCP Server \(AI Agent Integration\)](#)
 - 10. [Self-Hosting](#)
 - 11. [Troubleshooting](#)
 - 12. [Reference Links](#)
-

1. Introduction

Firecrawl is a web data API that lets you search, scrape, crawl, and interact with websites at scale, returning content in formats ready for AI systems —

Markdown, structured JSON, HTML, and screenshots. It handles the operationally hard parts of web scraping on the server side: rendering JavaScript-heavy pages,

rotating proxies, retrying failed requests, and respecting rate limits — so your application only has to deal with clean results.

It's available two ways:

- **Hosted API** — a managed service at `api.firecrawl.dev`, billed by credits.
- **Self-hosted / open source** — the same core engine, run on your own infrastructure.

Typical use cases include RAG pipelines, AI agents that browse the web, lead enrichment, deep research tools, and any workflow that needs structured data pulled from live pages.

2. Getting an API Key

1. Create an account at firecrawl.dev.

2. Generate an API key from your dashboard — it will look like `fc-xxxxxxx`.
3. Store it as an environment variable rather than hardcoding it:

```
export FIRECRAWL_API_KEY="fc-  
YOUR_API_KEY"
```

A limited **keyless** mode also exists for quick testing: it exposes Search, Scrape, and Parse only, is capped per IP address per day (by request count and by credit count), and does not support crawl, extract, map, or batch scrape. Signing up for a free key raises these limits and unlocks the full endpoint set.

3. Installation

Choose the SDK that matches your stack. All SDKs wrap the same REST API.

Python

```
pip install firecrawl-py
```

Node / TypeScript

```
npm install firecrawl
```

Go

```
go get  
github.com/firecrawl/firecrawl-go
```

Rust

```
cargo add firecrawl
```

Java and Elixir SDKs are also available. If you'd rather not use an SDK, every endpoint is a plain HTTPS REST call, so `curl` or any HTTP client works.

4. Authentication

Every request (aside from limited keyless usage) requires a Bearer token in the `Authorization` header:

```
curl -X POST
'https://api.firecrawl.dev/v2/scrape' \
  -H 'Authorization: Bearer fc-YOUR_API_KEY' \
  -H 'Content-Type: application/json' \
  -d '{ "url": "https://example.com" }'
```

SDKs read the key from the constructor argument or automatically from the `FIRECRAWL_API_KEY` environment variable.

5. Core Endpoints

5.1 Scrape

Fetches a single URL and returns it in whatever format(s) you request: Markdown, HTML, links, a screenshot, or schema-guided JSON.

```
from firecrawl import Firecrawl

app = Firecrawl(api_key="fc-
YOUR_API_KEY")
result =
app.scrape("https://example.com")
print(result.markdown)
```

Key options:

- `formats` — list of desired outputs (`markdown`, `html`, `links`, `screenshot`, `json`).
- `onlyMainContent` — strips navigation/footer boilerplate (enabled by default).
- `waitFor` — milliseconds to wait for JavaScript-rendered content before capturing the page.

- `actions` — a sequence of clicks, scrolls, or inputs to perform before scraping (for interactive pages).

Cost: 1 credit per page for a standard scrape; structured JSON extraction and enhanced proxy add extra credits.

5.2 Crawl

Starts from one URL and follows links to scrape an entire site or section. Crawl jobs are asynchronous — you get a job ID back and either poll for status, use a watcher, or receive a webhook when pages complete.

```
crawl_result = app.crawl(  
    "https://example.com",  
    limit=100,  
    scrape_options={"formats":  
        ["markdown"]}  
)  
  
for page in crawl_result.data:
```

```
print(page.metadata.source_url)
```

Key options:

- `limit` — maximum number of pages to crawl.
- `includePaths` / `excludePaths` — restrict the crawl to (or exclude) specific URL patterns.
- `maxDepth` — how many link-levels deep to follow from the start URL.
- `webhook` — an HTTPS endpoint Firecrawl will POST to as pages complete or the job finishes.

Crawl respects `robots.txt` rules for the `FirecrawlAgent` user agent. Before a job starts, Firecrawl checks that you have enough credits to cover the requested `limit`; if not, the request fails with a `402 Payment Required` and no job runs.

Cost: 1 credit per page scraped, same per-page add-ons as Scrape.

5.3 Map

Rapidly enumerates the URLs on a site without scraping full content — useful for deciding what to crawl before you spend credits on it.

```
mapped =  
app.map_url("https://example.com",  
limit=1000, search="docs")
```

A single map call can return up to 100,000 links and costs a flat 1 credit regardless of how many links come back.

5.4 Search

Runs a web search and, optionally, scrapes each result in the same call.

```
search_result = app.search(  
    "machine learning
```

```
site:github.com",  
    limit=10,  
    scrape_options={"formats":  
["markdown"]}  
)
```

Supports filtering by source type (web, news, images) and standard search parameters like language and country.

5.5 Extract

Pulls structured, schema-defined data out of one or many pages — the fastest route to clean typed output for a RAG or data pipeline.

```
from pydantic import BaseModel  
  
class Article(BaseModel):  
    title: str  
    author: str  
    published_date: str  
  
result = app.scrape(
```

```
"https://example.com/blog/post",  
    formats=[{"type": "json",  
"schema": Article}]  
)  
print(result.json)
```

You can also pass a natural-language prompt instead of (or alongside) a schema when the fields you need aren't fully known in advance.

Cost: JSON/structured extraction adds roughly 4 credits per page on top of the base scrape cost.

5.6 Interact / Agent

Lets you click, navigate, fill in forms, or otherwise operate a page programmatically or via a natural-language prompt, then extract the result — useful for pages that require interaction (logins, dropdowns, "load more" buttons)

before the content you want becomes available.

```
result =
app.scrape("https://example.com/st
ore")
scrape_id =
result.metadata.scrape_id
app.interact(scrape_id,
prompt="Search for wireless
headphones and open the first
result")
```

6. Output Formats

Format	Description
markdown	Clean, LLM-ready Markdown with boilerplate stripped
html	Cleaned or raw HTML

Format	Description
<code>links</code>	All links found on the page
<code>screenshot</code>	Full-page or viewport image capture
<code>json</code>	Structured data matching a schema or prompt

Requesting only the formats you need (e.g., `markdown` alone instead of `markdown + html + screenshot`) reduces processing time and, for some formats, credit cost.

7. Webhooks & Async Delivery

For long-running jobs like crawls, Firecrawl supports three delivery patterns:

1. **Polling** — periodically check job status with the returned job ID.
2. **Watcher** — an SDK helper that blocks and streams updates as they arrive.
3. **Webhooks** — Firecrawl sends an HTTP POST to a URL you provide whenever a page finishes, an error occurs, or the job completes. This is the best fit for production systems where results should land directly in a queue or database rather than being held in a running script.

```
crawl_result = app.crawl(  
    "https://example.com",  
    webhook={"url":  
"https://yourapp.com/webhooks/fire  
crawl"}  
)
```

8. Credits, Pricing & Rate Limits

Firecrawl bills by **credits** rather than flat request counts, so cost scales with what an operation actually does:

- Standard scrape: **1 credit/page**
- Batch scrape: **~0.5 credits/page**
- JSON/structured extraction: **+4 credits/page**
- Enhanced proxy: **+4 credits/page**
- PDF parsing: **+1 credit/PDF page**
- Map: **1 credit per call**, regardless of link count

Failed requests (site errors, timeouts) generally don't consume credits.

Rate limits are enforced per plan as requests-per-minute (RPM), with free tiers naturally lower than paid tiers, and enterprise plans supporting significantly

higher throughput plus higher concurrent-browser ceilings. Exceeding a limit returns an HTTP 429; exceeding your credit balance on a job that needs more than you have returns 402. If you need custom limits, Firecrawl's team can be reached to discuss enterprise plans.

Practical tips:

- Use batch/crawl endpoints instead of hammering /scrape in a loop for large jobs.
 - Request only the output formats you need.
 - Implement retry logic with exponential backoff on 429 responses.
 - Watch for 402 on crawl jobs — it means your remaining credits don't cover the requested page limit.
-

9. MCP Server (AI Agent Integration)

Firecrawl ships an official Model Context Protocol (MCP) server, letting you plug Firecrawl directly into Claude, Cursor, or any other MCP-compatible client without writing API glue code.

```
npx firecrawl-mcp
```

The MCP server exposes a multi-tool set (scrape, crawl, search, map, extract, parse, and more). A few integration notes:

- **File parsing** (PDFs, Word docs, spreadsheets) is handled by the `parse` tool — call it once per file rather than passing a list of files in one call.
- The hosted MCP endpoint can run **keyless** for Search, Scrape, and Parse only; other tools need an account connection or API key.

- Prefer JSON format with a schema over full Markdown when you only need specific fields — it keeps responses smaller and avoids overflowing the calling agent's context window.
-

10. Self-Hosting

Firecrawl's core engine and SDKs are open source (AGPL-3.0) and can run entirely on your own infrastructure via Docker Compose. This is a good option if you need full control over proxy configuration, data residency, or want to avoid per-credit billing at very high volume.

At a high level:

1. Clone the `firecrawl/firecrawl` repository.
2. Configure environment variables (Redis connection, optional proxy settings,

optional LLM keys for extraction features).

3. Run `docker compose up` to bring up the API, worker, and supporting services.
4. Point your SDK or REST calls at your self-hosted URL instead of `api.firecrawl.dev`.

Refer to the official self-hosting guide for current configuration details, since setup steps change as the project evolves.

11. Troubleshooting

Symptom	Likely cause / fix
Incomplete content on single-page apps	Increase <code>waitFor</code> , or use batch scrape, which allows more render time

Symptom	Likely cause / fix
Blocked by CAPTCHA or bot detection	Firecrawl handles common anti-bot measures but not all; try enhanced proxy options for stubborn sites
429 Too Many Requests	You've hit your plan's RPM or concurrency limit — reduce request rate or upgrade plan
402 Payment Required on crawl	Your credit balance doesn't cover the requested crawl limit — lower the limit or add credits
Crawl pulling too many irrelevant pages	Narrow scope with <code>includePaths</code> / <code>excludePaths</code> and <code>maxDepth</code>
Large crawl consuming	Use <code>map</code> first to preview the URL set, then scope the crawl deliberately

Symptom	Likely cause / fix
credits quickly	

12. Reference Links

- Documentation:
<https://docs.firecrawl.dev>
- API Reference:
<https://docs.firecrawl.dev/api-reference/introduction>
- Rate Limits:
<https://docs.firecrawl.dev/rate-limits>
- Core repo:
<https://github.com/firecrawl/firecrawl>
- Python SDK:
<https://github.com/firecrawl/firecrawl-py>
- MCP Server:
<https://github.com/firecrawl/firecrawl->

mcp-server

- Community:

<https://discord.gg/firecrawl>